



Собери API платежей на FastAPI

Небольшой сервис, через который онлайн-школа принимает оплату курса через Квитто: три эндпоинта и уведомление от банка. Кода немного, но в деньгах важна точность — копейки, повторные запросы и статусы.

Пиши с ИИ — Cursor, Claude Code, Copilot, чем тебе удобно. Мы так работаем сами. Смотрим не на то, сколько строк ты набрал руками, а на то, работает ли код и понимаешь ли ты каждую строку в нём.

СРОК

3 дня

С момента получения задания

ЧИСТОЕ ВРЕМЯ

≈ 3 часа

Больше не нужно — лучше меньше, но надёжно

СТЕК

FastAPI

Python 3.11+, SQLite или PostgreSQL

ИНСТРУМЕНТЫ

ИИ — да

Любые ассистенты, плюс файл AI_LOG.md

1 Стек

- Python 3.11+, FastAPI, Pydantic v2
- База — SQLite или PostgreSQL, через SQLAlchemy или другую ORM
- pytest + httpx
- Docker и миграции не обязательны — достаточно README с командой запуска

2 Предметная область

ЧТО	ПРАВИЛО
Тарифы	Создаются при старте: <code>basic</code> — 9 900 ₽, <code>standard</code> — 19 900 ₽, <code>premium</code> — 29 900 ₽.
Деньги	Все суммы в API и в базе — в копейках, целым числом: 19 900 ₽ → 1990000 . Никаких <code>float</code> .
Промокод	<code>KVITT010</code> даёт -10% от цены тарифа, регистр не важен. Неизвестный код — ошибка 422.
Способ оплаты	<code>card</code> , <code>sbp</code> или <code>installment</code> . Для рассрочки обязателен <code>installment_months</code> : 3, 6 или 12.
График рассрочки	Сумма к оплате делится на срок в копейках. Сумма всех платежей графика ровно равна сумме к оплате; лишние копейки — в первые платежи. Даты платежей не нужны.
Статусы	Новый платёж — <code>pending</code> . Допустимые переходы: <code>pending</code> → <code>succeeded</code> , <code>pending</code> → <code>failed</code> , <code>succeeded</code> → <code>refunded</code> . Остальные запрещены.

```
# Пример: standard без промокода, рассрочка на 3 месяца
amount = 1990000
schedule = [663334, 663333, 663333] # в сумме ровно 1990000
```

3 Эндпоинты

ЗАПРОС	ЧТО ДОЛЖНО ПРОИСХОДИТЬ
GET /tariffs	200 — список тарифов [{ id, title, price }] .
POST /payments	Тело: tariff_id , email , method , installment_months (для рассрочки), promo_code (необязательно). 201 — платёж создан. Если передан заголовок Idempotency-Key и платёж с таким ключом уже есть — вернуть его же с кодом 200, второй не создавать.
GET /payments/{id}	200 — платёж; 404, если его нет.
POST /webhooks/bank	Банк сообщает о смене статуса. Тело: payment_id , status . Платежа нет — 404. Запрещённый переход — 409 { error: "invalid_transition" } , статус не меняется. Успех — 200 { result: "ok" } .

Платёж в ответах: id , status , tariff_id , amount , discount , method , installment_months , schedule — список сумм в копейках или null , email , created_at .

Ошибки валидации — стандартный ответ FastAPI с кодом 422, свой формат не нужен.

4 Тесты

Минимум 5 тестов на pytest:

- сумма с промокодом и без, в том числе промокод в нижнем регистре
- график рассрочки: сумма графика равна сумме к оплате для 3, 6 и 12 месяцев
- повторный запрос с тем же Idempotency-Key не создаёт второй платёж
- запрещённый переход статуса возвращает 409 и не меняет статус
- запрос несуществующего платежа возвращает 404

5 Работа с ИИ — файл AI_LOG.md

ИИ можно и нужно использовать. Скрывать нечего — мы хотим увидеть, как ты с ним работаешь. В корне репозитория положи AI_LOG.md на полстраницы:

- какими инструментами и моделями пользовался;
- 2–3 ключевых запроса к ассистенту — как ставил задачу и какой контекст давал;
- где модель ошиблась или предложила слабое решение и как ты это заметил.

На созвоне с СТО попросим объяснить любой фрагмент кода и внести небольшую доработку вживую — с ИИ, при нас. Поэтому важно не только сдать работающий код, но и разобраться в нём.

6 Как сдавать

- Репозиторий на GitHub — публичный или приватный с доступом по приглашению
- README: как запустить, как прогнать тесты, пара примеров запросов curl
- AI_LOG.md в корне репозитория
- Ссылку отправить ответным сообщением в чат, откуда пришло задание

Бонусы — по желанию

- Запуск через `docker compose up`
- Проверка подписи вебхука: заголовок `X-Signature` — HMAC-SHA256 от тела запроса с секретом из переменной `WEBHOOK_SECRET` , неверная подпись — 401
- Миграции Alembic
- `GET /payments` с фильтрами по `email` и `status`
- GitHub Actions: `ruff` и `pytest` на каждый `push`
- Свои правила для ассистента в репозитории: `CLAUDE.md` , `.cursorrules` или аналог

7 Как будем оценивать — 100 баллов

КРИТЕРИЙ	БАЛЛЫ	ЧТО СМОТРИМ
Деньги и бизнес-логика	30	Копейки без потерь, промокод, график рассрочки, правила статусов
Повторы и статусы	15	Повтор с тем же ключом не создаёт дубль, запрещённый переход не меняет статус
Тесты	15	Обязательные сценарии покрыты, тесты проверяют результат, а не только код ответа
Код и структура	15	Понятная структура, типы, обработка ошибок
Работа с ИИ	15	Конкретный <code>AI_LOG.md</code> : осмысленные запросы, найденные ошибки модели, видно, что код проверен
Запуск и README	10	Проект запускается по инструкции с первого раза
Бонусы	+10	Сверх основной оценки

Не нужно: фронтенд, авторизация пользователей, Docker и миграции (если не берёшь бонусы), реальная интеграция с банком, деплой. Вопросы по заданию — пиши в тот же чат, отвечаем в течение рабочего дня.